

Informe Prueba Bimestral POO

Diego Javier Torres Auz,

Universidad Técnica Particular de Loja

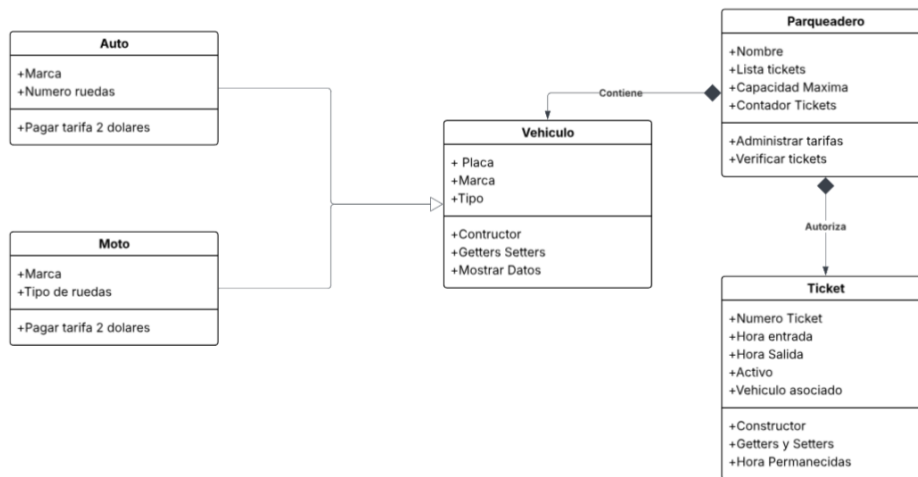
Programación Orientada a Objetos

Mgtr. Ramiro Leonardo Ramírez Coronel

28 de mayo del 2026

Diagrama UML

Sistema de Control y Facturación de un Parqueadero Automatizado



Código

Clase Main:

```
* @author daigo
*/
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Parqueadero p = new Parqueadero("Central", 10);

        System.out.println(" REGISTRO DE INGRESO ");

        System.out.println("Ingrese placa:");
        String placa = sc.nextLine();

        System.out.println("Ingrese marca:");
        String marca = sc.nextLine();
```

```
System.out.println("Ingrese tipo (Auto/Moto):");
String tipo = sc.nextLine();

System.out.println("Ingrese hora de entrada:");
int horaEntrada = sc.nextInt();

Vehiculo v = new Vehiculo(placa, marca, tipo);

p.registrarIngreso(v, horaEntrada);

sc.nextLine();

System.out.println("\n REGISTRO DE SALIDA ");

System.out.println("Ingrese placa del vehículo:");
String placaSalida = sc.nextLine();

System.out.println("Ingrese hora de salida:");
int horaSalida = sc.nextInt();
```

```
p.procesarSalida(placaSalida, horaSalida);

GestorArchivo.guardarEstado(p);

System.out.println("\nDatos guardados correctamente");
}
}
```

Clase Gestor de Archivos:

```
import java.io.*;

public class GestorArchivo {

    public static void guardarEstado(Parqueadero parqueadero) {

        try {

            ObjectOutputStream oos =
                new ObjectOutputStream(new FileOutputStream("parqueadero.dat"));

            oos.writeObject(parqueadero);

            oos.close();

            System.out.println("Estado guardado");

        } catch (Exception e) {
            System.out.println("Error al guardar");
        }

    }

}
```

```
    public static Parqueadero cargarEstado() {

        try {

            ObjectInputStream ois =
                new ObjectInputStream(new FileInputStream("parqueadero.dat"));

            Parqueadero p = (Parqueadero) ois.readObject();

            ois.close();

            return p;

        } catch (Exception e) {

            System.out.println("No existe archivo");

            return null;

        }

    }

}
```

Clase Parquero:

```
import java.io.Serializable;

public class Parquero implements Serializable {

    private String nombre;
    private Ticket[] listaTickets;
    private int capacidadMaxima;
    private int contadorTickets;

    public Parquero(String nombre, int capacidadMaxima) {
        this.nombre = nombre;
        this.capacidadMaxima = capacidadMaxima;
        listaTickets = new Ticket[capacidadMaxima];
        contadorTickets = 0;
    }

    public void registrarIngreso(Vehiculo veh, int horaEntrada) {

        if (contadorTickets < capacidadMaxima) {

            Ticket t = new Ticket(contadorTickets + 1, horaEntrada, veh);

            listaTickets[contadorTickets] = t;

            contadorTickets++;

            System.out.println("Ingreso registrado");
        } else {
            System.out.println("Parquero lleno");
        }
    }

    public Ticket buscarTicketPorPlaca(String placa) {

        for (int i = 0; i < contadorTickets; i++) {

            if (listaTickets[i].isActivo() &&
                listaTickets[i].getVehiculoAsociado().getPlaca().equals(placa)) {

                return listaTickets[i];
            }
        }

        return null;
    }
}
```

```

public void procesarSalida(String placa, int horaSalida) {

    Ticket t = buscarTicketPorPlaca(placa);

    if (t != null && t.isActivo()) {

        if (horaSalida < t.getHoraEntrada()) {
            System.out.println("Error: cambio de día no soportado");
            return;
        }

        t.setHoraSalida(horaSalida);

        int horas = t.calcularHorasPermanencia();

        double total = 0;

        if (t.getVehiculoAsociado().getTipo().equalsIgnoreCase("Auto")) {
            total = horas * 2;
        } else if (t.getVehiculoAsociado().getTipo().equalsIgnoreCase("Moto")) {

```

```

            total = horas * 1;
        } else if (t.getVehiculoAsociado().getTipo().equalsIgnoreCase("Moto")) {
            total = horas * 1;
        }

        if (horas > 5) {
            total = total - (total * 0.10);
        }

        t.setActivo(false);

        System.out.println("Horas: " + horas);
        System.out.println("Total a pagar: $" + total);

    } else {
        System.out.println("Ticket no encontrado");
    }
}

```

Clase Vehiculo

```
import java.io.Serializable;

public class Vehiculo implements Serializable {

    private String placa;
    private String marca;
    private String tipo;

    public Vehiculo(String placa, String marca, String tipo) {
        this.placa = placa;
        this.marca = marca;
        this.tipo = tipo;
    }

    public String getPlaca() {
        return placa;
    }
}
```

```
public void setPlaca(String placa) {  
    this.placa = placa;  
}
```

```
public String getMarca() {  
    return marca;  
}
```

```
public void setMarca(String marca) {  
    this.marca = marca;  
}
```

```
public String getTipo() {  
    return tipo;  
}
```

```
public void setTipo(String tipo) {  
    this.tipo = tipo;  
}
```

```
public void mostrarDatos() {  
    System.out.printf("Placa: %s\nMarca: %s\nTipo: %s\n",placa,marca,tipo);  
}  
}
```


Clase Ticket:

```
import java.io.Serializable;

public class Ticket implements Serializable {

    private int numeroTicket;
    private int horaEntrada;
    private int horaSalida;
    private boolean activo;
    private Vehiculo vehiculoAsociado;

    public Ticket(int numeroTicket, int horaEntrada, Vehiculo vehiculoAsociado) {
        this.numeroTicket = numeroTicket;
        this.horaEntrada = horaEntrada;
        this.vehiculoAsociado = vehiculoAsociado;
        this.activo = true;
    }

    public int calcularHorasPermanencia() {
        return horaSalida - horaEntrada;
    }
}
```

```
public int getNumeroTicket() {
    return numeroTicket;
}

public int getHoraEntrada() {
    return horaEntrada;
}

public int getHoraSalida() {
    return horaSalida;
}

public void setHoraSalida(int horaSalida) {
    this.horaSalida = horaSalida;
}

public boolean isActive() {
    return activo;
}
```

```

    public void setActivo(boolean activo) {
        this.activo = activo;
    }

    public Vehiculo getVehiculoAsociado() {
        return vehiculoAsociado;
    }
}

```

Salida:

```

    REGISTRO DE INGRESO |
Ingrese placa:
ABCD
Ingrese marca:
Toyoat
Ingrese tipo (Auto/Moto):
Moto
Ingrese hora de entrada:
3
Ingreso registrado

    REGISTRO DE SALIDA
Ingrese placa del vehiculo:
ABCD
Ingrese hora de salida:
9
Horas: 6
Total a pagar: $5.4
Estado guardado

Datos guardados correctamente

```

Deserialización archivo notas22.dat

Nombre completo: TORRES AUZ DIEGO JAVIER

acd: 3.1

aa: 1.4

ape: 3.3

Clase Vehiculo

En esta clase guardo los datos del vehículo como placa, marca y tipo
También hice el constructor y métodos para mostrar o modificar los datos

Clase Ticket

Esta clase representa el ticket del parqueadero
Aquí guardo la hora de entrada, salida, el estado del ticket y el vehículo asociado

También hice un método para calcular las horas que estuvo el vehículo

Clase Parqueadero

En esta clase hice la lógica principal del sistema

Aquí:

registro vehículos

busco tickets por placa

proceso la salida

calculo el valor a pagar

aplico descuentos

Clase GestorArchivo

Esta clase sirve para guardar la información del parqueadero en un archivo y recuperarla después usando serialización

Clase Main

En esta clase ejecuté el programa

Aquí pedí los datos por teclado con Scanner, registré ingresos y salidas de vehículos y guardé la información